

# How to Use Python?

*From primitives to a working hidden Markov model*

**Tim Lin**

y10124@princeton.edu

July 12, 2026



# Outline

Part 1 — Python 101

Part 2 — Dynamic Programming

Part 3 — Hidden Markov Model

## Part 1

---

# Python 101, a Guided Tour

# Roadmap for the tour

- ▶ Object model in code: identity, type, value
- ▶ Mutable vs. immutable — the aliasing trap
- ▶ Scalar types and strings
- ▶ Containers: `list`, `tuple`, `dict`, `set`
- ▶ Control flow: `for`, `while`, `enumerate`, `zip`
- ▶ Functions: arguments, defaults, the mutable-default gotcha
- ▶ Two Pythonic idioms: comprehensions, duck typing
- ▶ `import` and the `__main__` pattern
- ▶ Exercise (15 min)

# The object triple: identity, type, value

Every Python object has three things (from the morning deck, now in code):

```
x = [1, 2, 3]
id(x)      # identity: which object (its address)
type(x)    # type: what operations it supports  -> <class 'list'>
x          # value: the data/state it holds      -> [1, 2, 3]
```

- ▶ `a is b` → do the **names** point to the *same object*? (compares identity)
- ▶ `a == b` → do the objects have the *same value*? (compares value)

The distinction between `is` (identity) and `==` (value) is the #1 bug source for experimentalists. Predict both before you press enter.

# Mutable vs. immutable

```
x = [1, 2, 3]
y = x          # y is a NEW NAME for the SAME object
y.append(4)
print(x)       # -> [1, 2, 3, 4]    (!)  x changed too
```

```
x = 3
y = x
y = y + 1      # rebinds y to a NEW int object
print(x, y)    # -> 3 4            x is untouched
```

**Mutable** — changed in place:

list, dict, set.

**Immutable** — you rebind:

int, float, str, tuple, bool.

**Rule:** assignment **binds names**; mutation **changes objects**. “= is a label-maker, not a copier.”

# Scalar types and strings

```
n = 42          # int    (arbitrary precision in Python 3)
r = 3.14        # float  (IEEE-754 double)
flag = True     # bool   (a subclass of int: True == 1)
s = "spike"     # str    (immutable sequence of characters)

f"n={n}, r={r:.2f}"    # f-string -> 'n=42, r=3.14'
```

- ▶ Division:  $7 / 2 == 3.5$  (true),  $7 // 2 == 3$  (floor),  $7 \% 2 == 1$  (mod).
- ▶ Beware float equality:  $0.1 + 0.2 == 0.3$  is False.

Floats are finite-precision; equality tests on them are almost always a mistake — compare with a tolerance. (This foreshadows the **numerical-underflow** gotcha in the HMM.)

# Containers: the four workhorses

```
xs = [1, 2, 3]           # list - ordered, mutable
pt = (35.7, 139.8)       # tuple - ordered, immutable (good dict keys)
counts = {"a": 1, "b": 2} # dict - hash map, key -> value, O(1) lookup
seen = {1, 2, 3}         # set - unordered, unique, O(1) membership
```

Indexing and slicing (lists, tuples, strings):

```
a = [10, 20, 30, 40, 50]
a[0]      # 10      (0-indexed)
a[-1]     # 50      (negatives count from the end)
a[1:4]    # [20,30,40] (start inclusive, stop exclusive)
a[:2]     # [10,30,50] (step)
a[::-1]   # reversed
```

The `dict` is *the* workhorse: it powers namespaces/modules, and in an hour it becomes the `memoization cache` that turns exponential into linear.



# Control flow

```
for i in [1, 2, 3]:           # iterate over a sequence
    print(i)

for i, val in enumerate(xs):  # index + value together
    print(i, val)

for a, b in zip(names, scores): # iterate two sequences in lockstep
    ...

if score > 0.9:
    label = "high"
elif score > 0.5:
    label = "mid"
else:
    label = "low"

while not converged:         # loop until a condition
    step()
```

# Functions

```
def edit_cost(a, b, ins=1, dele=1, sub=1):    # pos + kwd defaults
    """Docstring: what the function computes."""
    return 0 if a == b else sub

edit_cost("A", "A")           # 0
edit_cost("A", "T", sub=2)    # 2    (call by keyword)
```

## Gotcha — mutable default arguments:

```
def bad(x, acc=[]):    # acc is created ONCE, shared across calls!
    acc.append(x); return acc

def good(x, acc=None):
    if acc is None: acc = []
    acc.append(x); return acc
```

The default `[]` is **one object bound at definition time**, not per call — the morning's names→objects model striking again. Fix: default `=None`, then create inside.

# Idiom 1 — comprehensions

```
squares = [x*x for x in range(10)]           # list comp
evens    = [x for x in xs if x % 2 == 0]      # with filter
lengths  = {w: len(w) for w in words}         # dict comp
pairs    = [(i, j) for i in range(3) for j in range(3)]
```

Reads as: “collect  $f(x)$  for each  $x$  in  $xs$  where  $p(x)$  holds.”

Comprehensions are the single most “Pythonic” construct and make research code dramatically shorter. [\[SAFETY NET\]](#) a plain for loop is *always* an acceptable substitute — comprehensions are for concision, not correctness.

## Idiom 2 — duck typing

```
def add_all(items):
    total = items[0]
    for x in items[1:]:
        total = total + x      # asks: what does '+' mean for THESE objects?
    return total

add_all([1, 2, 3])           # 6           (ints)
add_all(["a", "b", "c"])     # 'abc'       (strings concatenate)
add_all([[1], [2], [3]])     # [1, 2, 3]  (lists concatenate)
```

“If it walks like a duck and quacks like a duck...” — operations are resolved **at runtime** by the object’s type, not a declared type.

**Benefit:** generic code over many types.    **Cost:** type errors surface at **runtime**, not compile time — which is *why* modern code adds type hints (`def f(x: int) -> int`).

## import and the `__main__` pattern

```
import numpy as np          # find package, init once, bind name 'np'
np.array([1, 2, 3])         # attribute lookup on the module object

def main():
    data = load_data()
    train(data)

if __name__ == "__main__":  # runs only when executed directly,
    main()                  # NOT when imported
```

- ▶ Run directly (`python train.py`) → `__name__ == "__main__"` → `main()` runs.
- ▶ Imported (`import train`) → `__name__ == "train"` → `main()` does **not** run.

Importing binds a [module object](#) to a name; `np.array` is just attribute lookup. The guard stops expensive experiment code from firing accidentally on import.

## Exercise (15 min)

**Task:** Given a list of (state, observation) pairs, return a dict mapping each state to a count of observations seen in that state.

```
pairs = [("fair", "3"), ("loaded", "6"), ("fair", "1"), ("loaded", "6")]

def count_by_state(pairs):
    counts = {}
    for state, obs in pairs:
        counts.setdefault(state, {})
        counts[state][obs] = counts[state].get(obs, 0) + 1
    return counts
# expected: {'fair': {'3':1, '1':1}, 'loaded': {'6':2}}
```

**Advanced.** Stretch: rewrite the inner tally as a single `collections.Counter`, and/or vectorize the counts with `NumPy`.

## Part 2

---

# Dynamic Programming

# The core idea: exploit substructure

A problem is a good fit for **dynamic programming** when it has:

1. **Optimal substructure** — the solution is built from solutions to *subproblems*.
2. **Overlapping subproblems** — the *same* subproblems recur, so caching pays off.

**Dynamic programming** = recursion + memory.

Contrast with **divide & conquer**: there subproblems are *disjoint*, never repeat, so no cache is needed. DP is D&C plus a memory — used when the subproblem graph is a DAG, not a tree.



# Two ways to fill the memory

- ▶ **Top-down (memoization)**: write the natural recursion, cache each result the first time.
- ▶ **Bottom-up (tabulation)**: identify the dependency order, fill a table from base cases upward.

Same answers, same complexity; different code shape.

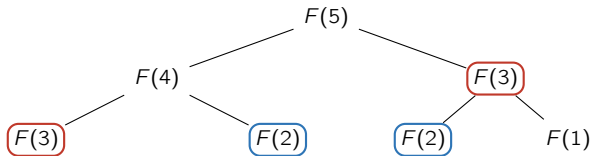
```
# top-down (memoization)
cache = {}
def f(state):
    if state in cache:
        return cache[state]
    cache[state] = combine(f(...))
    return cache[state]
```

```
# bottom-up (tabulation)
dp = [base, ...]
for state in order:
    dp[state] = combine(dp[...])
return dp[final]
```

## Example 1 — Fibonacci: seeing the overlap

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, F(1) = 1.$$

Naive recursion recomputes the same values exponentially often:



$F(3)$  and  $F(2)$  are recomputed on separate branches — that repetition is overlapping subproblems.

- ▶ Naive cost:  $O(\varphi^n)$ ,  $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$  (exponential).
- ▶ Memoized / tabulated:  $O(n)$  time,  $O(1)$  space bottom-up.

Caching turns an exponential tree into a linear chain.

## Example 1 — Fibonacci: the code

```
# top-down (memoization)
def fib(n, cache={}):
    if n < 2:
        return n
    if n not in cache:
        cache[n] = (fib(n-1, cache)
                    + fib(n-2, cache))
    return cache[n]
```

```
# bottom-up (tabulation), O(1) space
def fib_bu(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

The mutable-default `cache={} here is the intended shared-state version — callback to the earlier gotcha slide: same mechanism, now used on purpose.`

**Notebook checkpoint.** **Checkpoint 1:** run both, time naive vs. memoized for `n = 35`.

## Example 2 — Edit distance: the problem

**Edit distance**  $D(a, b)$  = minimum number of single-character *insertions*, *deletions*, or *substitutions* to turn string  $a$  into string  $b$ .

Example: kitten  $\rightarrow$  sitting has edit distance **3**  
(k $\rightarrow$ s substitute, e $\rightarrow$ i substitute, insert g).

Why it's a step up from Fibonacci:

- ▶ state is *2-dimensional* (a prefix of  $a$  and a prefix of  $b$ ),
- ▶ the recurrence has a genuine *choice* (min over three operations),
- ▶ we'll want to recover *which* edits — the solution, not just its cost.

## Example 2 — Edit distance: the recurrence

Let  $D[i][j]$  = edit distance between prefixes  $a[1:i]$  and  $b[1:j]$ . Base cases (turn a prefix into the empty string):  $D[i][0] = i$ ,  $D[0][j] = j$ .

$$D[i][j] = \begin{cases} D[i-1][j-1], & a_i = b_j \quad (\text{free match}) \\ 1 + \min \begin{cases} D[i-1][j] & (\text{delete } a_i) \\ D[i][j-1] & (\text{insert } b_j) \\ D[i-1][j-1] & (\text{substitute}) \end{cases} & a_i \neq b_j \end{cases}$$

Answer:  $D[m][n]$ . Complexity:  $O(mn)$  time and space.  $D[i-1][j-1]$  is needed by three different cells — that **table reuse** is the DP payoff.

## Example 2 — Edit distance: table + traceback

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    D = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): D[i][0] = i
    for j in range(n+1): D[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            if a[i-1] == b[j-1]:
                D[i][j] = D[i-1][j-1]
            else:
                D[i][j] = 1 + min(D[i-1][j],          # delete
                                D[i][j-1],          # insert
                                D[i-1][j-1])        # substitute

    return D[m][n]
```

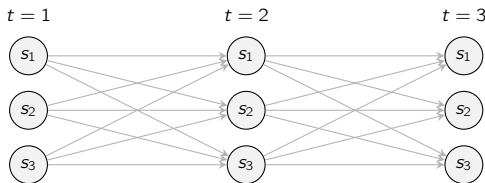
**Notebook checkpoint.** **Checkpoint 2:** fill the table for kitten/sitting, print it as a grid.

## Example 3 — Trellis: DP over time

Now the subproblems are indexed by **time** and **state**. Setup:  $T$  time steps (“layers”),  $K$  states per step.

- ▶  $c_t(j)$  = cost of being in state  $j$  at time  $t$  (**node cost**).
- ▶  $w(i, j)$  = cost of moving from state  $i$  to state  $j$  (**transition cost**).

Goal: the minimum-cost path that picks one state per time step.



each edge has cost  $w(i, j)$ ; each node has cost  $c_t(j)$ .

## Example 3 — Trellis: the recurrence

Let  $V_t(j)$  = cost of the best path that **ends at state  $j$  at time  $t$** . Base case:  $V_1(j) = c_1(j)$ .

$$V_t(j) = c_t(j) + \min_i [V_{t-1}(i) + w(i, j)].$$

Answer:  $\min_j V_T(j)$ , with backpointers  $\text{bp}_t(j) = \arg \min_i [\dots]$  to recover the path.

Complexity:  $O(TK^2)$  ( $T$  columns  $\times K$  targets  $\times K$  sources) — versus  $K^T$  paths if enumerated. The inner  $\min_i$  reuses the whole previous column  $V_{t-1}$ : the **overlapping-subproblems** payoff again.

**Notebook checkpoint.** Checkpoint 3: implement `min_cost_path`



## Part 2 wrap: what you now hold

- ▶ Recognize DP: optimal substructure + overlapping subproblems.
- ▶ Fill it two ways: top-down memo or bottom-up table.
- ▶ Recover solutions, not just costs, with backpointers.
- ▶ The trellis recurrence,  $V_t(j) = c_t(j) + \min_i [V_{t-1}(i) + w(i, j)]$ , is one operator-swap away from HMM inference.

Where DP shows up in your world: sequence alignment (genomics), Viterbi (this afternoon), the Bellman equation of reinforcement learning, CTC in speech models.

## Part 3

---

# Mini-Project: A Hidden Markov Model

# The setup: hidden states, visible observations

A system moves through **hidden states**  $z_{1:T}$ ,  $z_t \in \{1, \dots, K\}$ , emitting **observations**  $x_{1:T}$  we can see.

Running example — **the occasionally-dishonest casino:**

- ▶ hidden state  $z_t \in \{\text{fair}, \text{loaded}\}$  (we can't see which die),
- ▶ observation  $x_t \in \{1, \dots, 6\}$  (the roll we *do* see),
- ▶ the casino sometimes swaps dice between rolls.

Inference question: given the rolls, **when was the loaded die in play?**

# The model: three parameter groups $\theta = (\pi, A, B)$

$\pi_i = P(z_1 = i)$  — initial state distribution (where it starts)

$A_{ij} = P(z_t = j \mid z_{t-1} = i)$  — transition matrix ( $K \times K$ , how states switch)

$B_i(x) = P(x_t = x \mid z_t = i)$  — emission model ( $K \times M$ , how a state generates data)

Two defining assumptions:

- ▶ **Markov property:**  $z_t \perp z_{1:t-2} \mid z_{t-1}$  (future depends on the past only through the present state).
- ▶ **Emission independence:**  $x_t \perp \text{everything} \mid z_t$  (an observation depends only on its own hidden state).

Everything downstream — the factorization and the  $O(TK^2)$  efficiency — comes from these two conditional independences.

# The model: joint factorization

Under the two assumptions, the joint probability of a full trajectory factorizes:

$$P(z_{1:T}, x_{1:T}) = \pi_{z_1} B_{z_1}(x_1) \prod_{t=2}^T A_{z_{t-1}z_t} B_{z_t}(x_t).$$

This is the generative story: pick  $z_1$  from  $\pi$ , emit  $x_1$  from  $B_{z_1}$ , transition via  $A$ , emit again, ...

**Notebook checkpoint.** `sample_hmm(pi, A, B, T)` is provided — read it to see this generative story in code.

# Three questions you can ask an HMM

(Rabiner's framing, 1989)

1. **Evaluation** — how likely is this data?  $P(x_{1:T} | \theta) \rightarrow$  **forward algorithm**
2. **Decoding** — what hidden path best explains it?  $\arg \max_z P(z | x) \rightarrow$  **Viterbi**
3. **Learning** — optimizing parameters  $\arg \max_{\theta} P(x | \theta) \rightarrow$  **Baum–Welch (EM)**

Today: we build **evaluation** and **smoothing** (forward–backward).

# Why naive evaluation is hopeless

$$P(x_{1:T}) = \sum_{z_{1:T}} P(z_{1:T}, x_{1:T})$$

sums over  $K^T$  possible state sequences — exponential.

But it's the **same trellis**: replace “cost” with “probability” and min with  $\sum$ , and it's  $O(TK^2)$ .  $K^T$  is astronomical ( $2^{300}$  for the casino with  $T = 300$ ); the forward algorithm computes the exact sum in linear time by reusing  $\alpha_{t-1}$  instead of re-walking every path.

# Forward-backward Algorithm

Define the **forward variable**  $\alpha_t(i) \equiv P(x_{1:t}, z_t = i)$  and the **backward variable**  $\beta_t(i) \equiv P(x_{t+1:T} \mid z_t = i)$

$$\alpha_1(i) = \pi_i B_i(x_1)$$

$$\alpha_t(j) = B_j(x_t) \sum_i \alpha_{t-1}(i) A_{ij}$$

$$P(x_{1:T}) = \sum_i \alpha_T(i) \beta_T(i) = 1$$

$$\beta_t(i) = \sum_j A_{ij} B_j(x_{t+1}) \beta_{t+1}(j)$$



# Smoothing posterior — the thing we actually want

$$\gamma_t(i) \equiv P(z_t = i \mid x_{1:T}) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)}$$

“Given the **whole** recording, what state was the system in at time  $t$ ?”

- ▶  $\alpha_t(i)$  carries evidence from the **past** ( $x_{1:t}$ ),
- ▶  $\beta_t(i)$  carries evidence from the **future** ( $x_{t+1:T}$ ),
- ▶ their product, normalized, is the state posterior at each  $t$ .

**Notebook checkpoint.** This is what your plot will show:  $\arg \max_i \gamma_t(i)$  overlaid on the true hidden states.

## Bridge back to Part 2: Viterbi is your trellis

Set the trellis costs to negative log-probabilities:

$$c_1(j) = -\log \pi_j - \log B_j(x_1), \quad c_t(j) = -\log B_j(x_t), \quad w(i, j) = -\log A_{ij}.$$

Then the min-cost path  $V_t(j) = c_t(j) + \min_i [V_{t-1}(i) + w(i, j)]$  **is** the most-likely state sequence (Viterbi). Products of probabilities become sums of costs; “most likely” becomes “cheapest.”

**Advanced.** The forward algorithm is the *same* recursion with  $\min \rightarrow \text{logsumexp}$ . Working in log space also pre-empts underflow (next slide).

# Two gotchas

**1. Numerical underflow.** Products of many probabilities  $\rightarrow 0$  fast (recall  $0.1+0.2$  from the morning).

- ▶ *Pragmatic fix (recommended today):* normalize  $\alpha_t$  to sum to 1 each step (Rabiner scaling); the summed logs of the normalizers give  $\log P(x_{1:T})$  for free.
- ▶ *Cleaner fix:* do everything in log space with `logsumexp`.

**2. Smoothing  $\neq$  decoding.** The pointwise-MAP states  $\arg \max_i \gamma_t(i)$  can differ from the jointly-MAP Viterbi path — the sequence of individually-most-likely states need not be a coherent (or even legal) path.

# The coding session (35 min, scaffolded)

**Provided:** `sample_hmm(pi, A, B, T)` (generator) and a plotting cell (true vs. inferred states). **You write ( $\approx 14$  lines total):**

```
def forward(pi, A, B, x):      # ~6 lines: alpha recursion (+ scaling)
    ...
def backward(A, B, x):        # ~6 lines: beta recursion
    ...
def posterior(alpha, beta):    # ~2 lines: gamma = normalize(alpha*beta)
    ...
```

**Then:** simulate  $\rightarrow$  infer  $\rightarrow$  plot, and watch  $\arg \max_i \gamma_t(i)$  track the true hidden states. Because the data is synthetic, ground truth is known and correctness is visible in the plot. Check  $\log P(x_{1:T})$  against `hmmlearn`.

# Extensions

For anyone who finishes early:

- ▶ **Viterbi with backpointers** — decode the single best path; compare it to the  $\gamma$ -argmax path (illustrates gotcha 2).
- ▶ **One Baum–Welch (EM) update** — *learn*  $\theta$  from data. The forward–backward quantities  $\alpha, \beta, \gamma$  are exactly the E-step sufficient statistics.
- ▶ **Gaussian emissions** — swap the categorical  $B$  for a Gaussian; re-run on a continuous toy signal.

# Where this lives in neuroscience

- ▶ **Event segmentation:** Baldassano et al. (2017, *Neuron*) fit an HMM to fMRI to find event boundaries in continuous experience — hidden state = “which event.”
- ▶ **Behavioral states:** the GLM-HMM (Ashwood et al., 2022, *Nat. Neuro.*; IBL) models animals switching between engaged/disengaged strategies.
- ▶ **Decoding with temporal structure:** an HMM is what single-trial neural decoding becomes when you add a state-transition prior instead of treating timepoints independently.

# References / pointers

- ▶ Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms* (CLRS), ch. 15 — dynamic programming.
- ▶ Erickson, *Algorithms* (free online), ch. 3 — DP, with the substructure framing.
- ▶ Rabiner (1989), “A Tutorial on Hidden Markov Models...”, *Proc. IEEE* — the classic forward–backward/Viterbi reference.
- ▶ Bishop, *Pattern Recognition and Machine Learning*, ch. 13 — HMMs as graphical models; ch. 8 — sum-product/max-product.
- ▶ Durbin, Eddy, Krogh, Mitchison (1998), *Biological Sequence Analysis* — the casino, and edit distance  $\leftrightarrow$  alignment.
- ▶ Baldassano et al. (2017), *Neuron*; Ashwood et al. (2022), *Nature Neuroscience* — neuro applications.